

Introductions to Big Data Systems: Homework 3

Francisco Diego Ariel Romero Contreras

法兰西思科

2025403471

francisco.romero.c@ug.uchile.cl

Introduction

In this homework we were asked to implement a “array per process sum reduce” using the MPI API, the goal is to simulate the following MPI_Reduce.

```
void Example_MPI_Reduce(const int *sendbuf, int *recvbuf, int count){
    MPI_Reduce(sendbuf, recvbuf, count, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
}
void YOUR_Reduce(const int *sendbuf, int *recvbuf, int count)
```

Where taken N process (where N can be 2 or 4), each has an array of large count, the reduction consist in a sum by index that return an array.

For example for 4 process each with an array:

A [3 4 2 4]

B [1 2 3 6]

C [5 3 2 3]

D [4 4 2 1]

The reduction ends like this:

R [13 13 9 14]

The API call **permitted** are:

- MPI_Send
- MPI_Recv
- MPI_Sendrecv
- MPI_Isend
- MPI_Irecv
- MPI_Comm_size
- MPI_Comm_rank
- MPI_Barrier

The **Prohibited** are:

- MPI_Reduce
- MPI_Allreduce

Solution

The reduction can be made with a binary procedure, where half of the nodes receive an array from another process and the other half make the sum between the local array and the external, later the process that have the sum of arrays continue with the sum.

So, the solution seems the following way, respectfully 2 and 4 cores.

Operation	process 0	process 1
Array	A [3 2 5 1]	B [5 6 3 8]
rank%step == 0?	T	F
Send/Recv	Recv array from 1	Send array to 0
Sum	A [3+5 2+6 5+3 1+8]	-
End	Return A [8 8 8 9]	-

Table 1: Execution for 2 process

Operation	process 0	process 1	process 2	process 3
Array	A [5 2 3 1]	B [6 3 4 1]	C [9 3 5 1]	D [1 6 6 4]
rank%step==0?	T	F	T	F
Send/Recv	Recv array from 1	Send array to 0	Recv array from 3	Send array to 2
sum	A [5+6 2+3 3+4 1+1]	-	C [9+1 3+6 5+6 1+4]	-
Array	A [11 5 7 2]	B [6 3 4 1]	C [10 9 11 5]	D [1 6 6 4]
rank%stetp==0?	T	F	F	F
Send/Recv	Recv array from 2	Send failed	Send array to 0	Send failed
sum	A [11+10 5+9 7+11 2+5]	-	-	-
End	Return A [21 14 18 7]	-	-	-

Table 2: Execution for 4 process

Performance

This solution doesn't seem as the optimal, as there are moments where some process are IDLE, this can be a problem for the case 2 process and large input, where the parallelization is not possible because one process has to send information and the other the reception and sum.

For 2 process at the range [0, 1M] does a bad performance, doing the double of time in this range, in other ranges it doesn't performs as well that the MPI implementation.

For 4 process at the range [0, 1M] it has a better performance that MPI, at the range [16M, 256M] it achieve a slight worse performance that MPI.

elements	2P MPI_Reduce	2P Your_Reduce	4P MPI_Reduce	4P Your_Reduce
64K	495	965	2091	1372
1M	6285	11802	15275	15100
16M	75295	114697	149885	166933
256M	1105856	1670364	2229542	2442406

Table 3: Performance of the Reduce algorithm 'Your_Reduce' vs 'MPI_Reduce' with 2 and 4 process, measurement in μ s

Conclusions

The implementation for 2 process clearly has a different approach, where the MPI reduce has better times in comparative with the student's reduce. For 4 process the student's implementation achieve better times for elements under 1M elements, and a slight worse performance as added more elements.

So the MPI reducer might have a complete different approach for a 2 process case where the cores have a better balance, but probably for the 4 process case the MPI have a higher base cost for adjust the communications.